Do it Kodamai Style!

Typed interfaces for agentic AI — and why they're the same good idea programmers have trusted for decades.

Agentic AI is everywhere, and almost everyone is building it the same way: wire a language model into a workflow and hope. Kodamai does it differently. Here is the idea from the ground up — and if you have ever trusted a type-checker, you already understand why it works.

1. Types, and why programmers trust them

The first programming languages were untyped: a program was just a list of instructions, and the only way to learn whether it made sense was to run it and watch it break. Then came **types**. Once you declare

```
plus :: Int -> Int -> Int
```

the machine knows `plus` takes two integers and returns one — so it can reject `"apple" plus 5` the moment you write it, without ever running the program. The mistake is caught at **compile time**, not at run time; before the code ships, not at 2 a.m. in production. That one move — say what a thing is, and let the machine check it — is among the most valuable ideas in all of software.

Correct by construction. Basic types catch nonsense, but you can ask for far more. Sharpen the types until they give a full *specification* of what the code should do; then type-checking either shows the code fails to meet it, or hands you a mathematical guarantee that it does. Compare:

```
sort :: List Int -> List Int
```

This promises only that the output is *a list of integers* — a partial guarantee; a broken `sort` that returned its input untouched would still type-check. Now sharpen it:

```
correctSort :: List Int -> SortedList
```

The return type now says *sorted*. If this type-checks, the output is guaranteed sorted — established by the compiler, before the program ever runs. Not tested, not hoped for: guaranteed. That is what turns code into **trustable, safe software**.

Scale by composition. Types also let software scale, because they let programs slot together like Lego bricks. A brick connects only where the studs line up; a typed program connects only where the types line up — the **preconditions** of one are exactly met by the **post-conditions** of another. Guarantee every join and you can assemble vast systems from small pieces with confidence.

Adapt by substitution. And because a connection is defined by the type, not by the particular piece, you can pull any component out and drop in another that fits the same typed hole. Swap one brick for a better brick of the same shape and the structure still stands — nothing around it breaks.

Compile-time safety, correctness by construction, scalable composition, effortless substitution: decades of programming have shown these are exactly what good types buy you.

> **THE DEEPER POINT**
>
> The type has quietly done something profound: it **internalises the specification** — until now written only in natural language — **into the logic**, as the type of the program. The specification is thereby a **logical entity**; and the code is a **mathematical entity** too. Living in one formal world, the two can be fed into **mathematical reasoning together** — so the machine can *prove*, not merely test, that the code meets its specification.

Let's use this amazing technology for AI agents.

2. Everything is an interface (P, R)

The simplest picture of an agent is a function `A : T → T'` — given an input of type `T` it returns an output of type `T'`. That is the **functional-programming** view, and it is fine as far as it goes. But agents are fundamentally **reactive**: they accept **events** and produce **responses**, and which responses even make sense depends on the event. To take that seriously we adopt an **event-driven, process-theoretic** view of AI, built on **containers**.

> **DEFINITION — A CONTAINER**
>
> A **container** is a pair
>
> `(P : Type, R : P → Type)`
>
> — a type `P` of **prompts**, together with a family `R` assigning to each prompt `p : P` the type `R p` of **permitted responses**.

An agent's interface is exactly such a container: `P` is the **type of prompts** (events) it accepts, and for each prompt `p`, `R p` is the **type of responses** it may return. Because `R` depends on `p`, different prompts permit different response types.

The thing you actually want is an inhabitant of

```
(p : P) → R p
```

a function that, given any prompt of type `P`, returns a response of type `R p`. That single shape covers a lot — in every case `P` is a type of prompts and `R p` a type of responses: an **agent** (prompts are requests; responses, the replies it may give); a **task or action** (prompts are tasks; responses, their valid outcomes); **theorem proving** (prompts are propositions; responses are proofs of `p` — so `(p:P) → R p` *is* a prover); **questions and answers** (prompts are questions; responses, the answers that count as correct).

This is really about trust. A raw LLM has type `String → String`: hand it any string, it hands back any string — nothing constrains what you may ask or what it may say. A typed interface states, precisely, *which prompts are accepted and only which replies are tolerated.* So `(p:P) → R p` is the trustworthy version of "call the model": it can only be asked well-formed things, and it can only answer with things you have declared acceptable. Trust stops being a hope about behaviour and becomes a property of the type.

THE DEEPER POINT

This is correct by construction, now for agents. An interface `(P, R)` is an agent's specification **internalised as a type**: it fixes exactly which prompts may be asked and, for each, exactly which responses are permitted. An agent's remit is therefore a **logical object**, and "may it do that?" becomes a question the **compiler answers** — anything outside the type is a compile-time error, not a production incident. Typed LLMs — models that can themselves be checked against such constraints — are under development. This is doing LLMs, **Kodamai Style!**

3. You rarely build the answer directly — you delegate

Writing `(p:P) → R p` directly for a rich `P` — say, "handle any incoming invoice" — is hard. So you don't. You implement a hard, **high-level** interface `(P, R)` by leaning on an easier, **low-level** one `(P', R')`, together with a translation between them. That translation is a **container morphism**.

DEFINITION — A CONTAINER MORPHISM

A morphism `(P, R) → (P', R')` is a pair of maps

```
f : P → P'        g : (p : P) → R'(f p) → R p
```

— a **delegation** `f` that turns a **high-level** prompt into a **low-level** one, and an **amalgamation** `g` that turns the low-level response back into the high-level one. (Responses flow *backwards*, and how they combine may depend on the original high-level prompt `p`.)

Delegate down, amalgamate up. That is an agent's entire strategy for answering.

The goal, stated exactly: a morphism into `(1,1)`. Let `(1,1)` be the trivial interface — one prompt, one response. A container morphism `(P, R) → (1,1)` unfolds to precisely `(p:P) → R p`: *answering is a morphism into* `(1,1)`. So a **typed LLM** is a morphism `(P, R) → (1,1)` — a model wrapped so that it accepts only prompts of type `P` and answers only with responses of type `R p`; and a **general LLM** is `(String, String) → (1,1)` — the degenerate case where both the prompt type and the response type are just `String` (any string at all). Kodamai's job is to move you from the second to the first. **Open agents.** A morphism all the way into `(1,1)` is an **agent** in the ordinary sense: handed a prompt, it just answers. But a morphism need not bottom out. A morphism `(P, R) → (P', R')` into a *non-trivial* interface is what we call an **open agent**: rather than answering outright, it exposes an open slot `(P', R')` for another component to fill. Those slots are exactly how we **slot agentic components together** — match one component against the `(P, R)` and `(P', R')` of the next, and the types guarantee they fit. Closed agents (into `(1,1)`) are where a computation bottoms out; open agents are the building blocks you compose.

And this is just continuations. Look again at implementing `(P, R)` through `(P', R')` and a morphism: `f` sets up a sub-question, and `g` is *what you do with its answer* — a continuation. Delegation installs the continuation; amalgamation runs it. The whole "container morphism" story is continuation-passing for typed agents: ask a simpler question, then continue with its answer, bottoming out when you finally answer directly — a morphism into `(1,1)`.

THE DEEPER POINT

None of this needs heavy machinery, but the structure is what makes the pieces fit. The interfaces `(P, R)` are **objects**, the container morphisms are the **arrows** between them, they **compose**, and `(1,1)` is the **terminal object** where every answer lands — a category (this one is called *Poly*). You need not care about the name; the payoff is concrete: your agents, tasks, provers and Q&A components all live in **one algebra of composition**, and the mathematics tells you exactly when and how they connect — **before you run anything**.

4. Containers and Agentic Orchestration

One agent is rarely the whole story: real systems are **many** agents wired into **workflows**. The elegant part is that the wiring needs no new machinery — the ways of combining agents are simply **operators on containers**. Each operator takes interfaces and returns a new interface, so a workflow is a **term** built from these operators; and because the result is again a container, **a whole workflow is itself an agent**. Four operators do most of the work (write a container as `(P, R)`).

4.1 Parallel composition

Run two processes side by side — ask both, and take both answers.

$$(P, R) \otimes (P', R') = (P \times P', R \times R')$$

A prompt is a pair `(p, p')` — one for each — and a response is a pair `(r, r')`: both are asked at once, and both answer. This is fan-out to independent specialists — `VAT` ⊗ `Cashflow` ⊗ `Anomaly`, all consulted together.

4.2 Branching: the coproduct

Accept either kind of prompt.

$$(P, R) + (P', R') = (P + P', [R, R'])$$

A prompt is tagged *left* (a `p : P`) or *right* (a `p' : P'`) and carries the responses of whichever side it came from. Branching is then a morphism `(P, R)` → `(P', R') + (P'', R'')`: its **delegation** `f : P → P' + P''` inspects each high-level prompt and **routes** it left or right, and its amalgamation handles the response of whichever branch was taken — value-dependent routing, `if approvable? then AutoApprove else Approvals`.

4.3 The product

Ask two different sub-agents to solve a problem, and accept an answer from either of them.

$$(P, R) \times (P', R') = (P \times P', R + R')$$

A prompt is a pair `(p, p')` — you pose the problem to *both* sub-agents — but a response is just *one* of the two: an `r : R p` *or* an `r' : R' p'`. The product therefore runs two sub-agents and accepts whichever answers — a race, or redundancy for reliability. Where the coproduct is a choice of *which prompt arrives*, the product is a choice of *which answer to accept*.

4.4 Sequential composition

Do one process, then — depending on its answer — do the next.

$$(P, R) \triangleleft (P', R') = (\Sigma\ p{:}P.\ (R\ p \to P'),\ \ \Sigma\ r{:}R\ p.\ \ R'(k\ r))$$

A prompt is a first prompt `p : P` together with a rule `k : R p → P'` that turns the first response into the second prompt; a response pairs the first answer `r : R p` with an answer in `R'(k r)` to the second prompt it triggered. This is prompt-chaining — `Intake ◁ Ledger ◁ Reconcile`, each stage feeding the next.

> **THE DEEPER POINT**
>
> Agentic orchestration and agentic workflows are **formally definable as operators on containers** — parallel, branching, product, sequential, each an operation taking interfaces to interfaces, each with a precise definition. A workflow is a **term** in this algebra, and the whole term is itself a container: *another agent*. Orchestration stops being glue code and hope and becomes **algebra**. **Agentic AI has become logic!**

Summary

Types are the most amazing technology in programming languages: so powerful, and so universal, that people have almost forgotten they are there. Kodamai is dedicated to transferring that power to AI agents — and we have only scratched the surface of what a container-based type system can do for them.

As Park Jae Sang would no doubt say:

Do it Kodamai Style!